

Week 6 Updates

Summary

- Fixed architecture from last meeting to be Lorentz-equivariant
- Got exploding outputs (outliers on the order of 10^{20}) under control
- Set up distributed training (HF accelerate)
- Fixed bug with scaling of jet attributes ($O(100 - 3000)$ to $O(1)$ for all features))
- Continue to have issues with the model learning the trivial mapping

Current architecture I

1. Embed scalar time value to random Fourier features through two fully connected layers with 32 and 64 nodes (following [1])
2. Concatenate time embedding with jet conditioning information, which consists of one-hot encoded type, η , mass, p_T , number of constituents
3. Pass this through a fully connected layer of size 64 to produce the initial global embeddings g^0
4. Concatenate g^0 with features $x_1^0 \dots x_N^0$ initialized from the initial distribution p_0 .
5. Now, we use L Lorentz-group equivariant layers.
 - 5.1 The input to the $l + 1$ -th layer is g^l and $x_1^l \dots x_N^l$.
 - 5.2 Calculate messages $m_{ij}^l = \phi_e^l(\psi(\|x_i^l - x_j^l\|), \psi(\langle x_i^l, x_j^l \rangle))$
 - 5.3 **Update the global embedding**

$$g_{l+1} = \phi_g^l(g^l, \alpha_l \sum_{i,j \in [N]} \phi_m^l(m_{ij}) \cancel{m_{ij}^l})$$

Current architecture II

5.4 Feed in the updated global embedding as well as the original embedding to update the graph embeddings

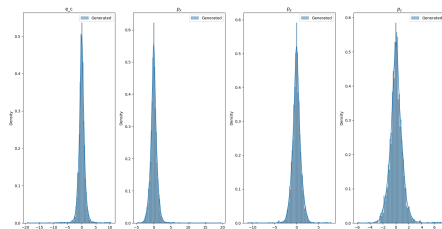
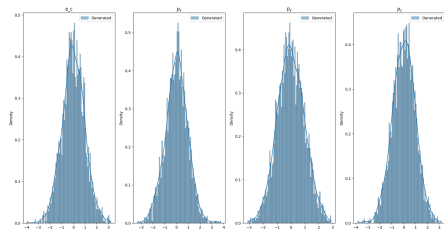
$$x_i^{l+1} = x_i^l + \gamma_l \sum_{j \in N, j \neq i} \phi_x^l(g^0, g^{l+1}, m_{ij}) x_j^l$$

5.5 Pass the results g^{l+1} and x_i^{l+1} to the next layer

6. **Final output** $v_\theta(t, x^0)_i = x_i^L$

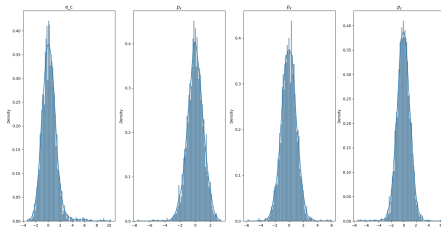
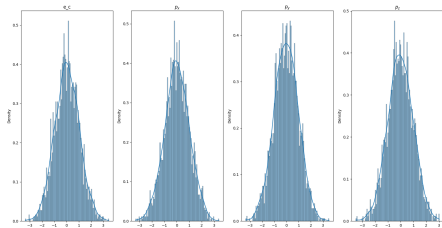
- ϕ = MLP with LeakyRELU activations except for ϕ_x , which uses tanh (for expressiveness in linear combinations)
- α initialized to $\frac{1}{N^2}$, γ initialized to $\frac{1}{N}$

Results



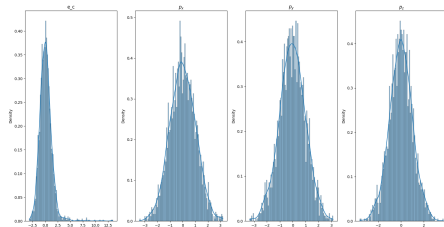
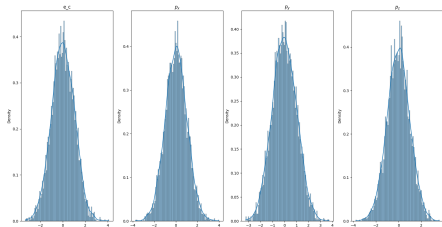
Same results with simple, non-LE MLP

Only use time embedding and jet type to generate global embedding, pass through 6 layers with LeakyReLU activations



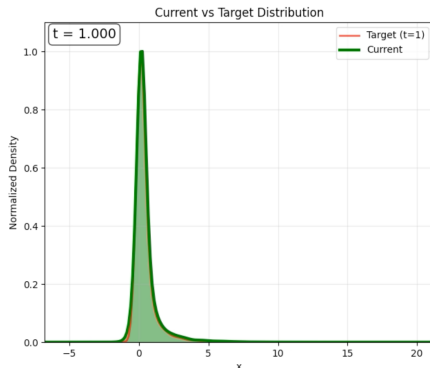
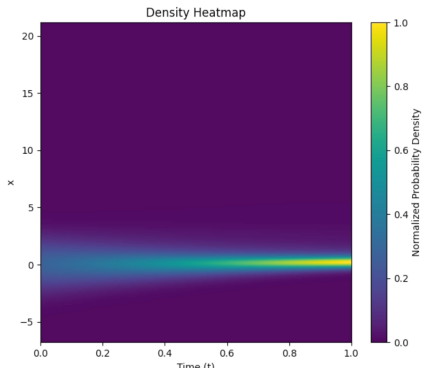
Train MLP only E/c

Maintain 150-dimensional particle cloud structure but discard p_x, p_y, p_z in training



Train unconditional MLP on 1-D flattened version of E/c

Remove jet attributes, flatten particle clouds (removing all zero-padded particles).
Extract 5000 E/c values and train a simple MLP (from week 1)



Hypotheses

- Training difficulty or programming bugs with masking
- Time embedding not being transmitted through: updates are roughly the same for all timesteps
- ~~Initial or intermediate particle clouds don't form a basis in 4-momenta space~~

Ideas

- Slightly refined architecture
 - Transmit time embedding and global embedding separately, i.e., replace g^L with g^L, t_{emb} and g^0 with g^0, t_{emb}
 - $m_{ij}^l = \phi_e(\|x_i^l - x_j^l\|, \langle x_i^l, x_j^l \rangle, \mathbf{g}^1, \mathbf{g}^0)$
- Break LE in the last step
 - LorentzNet stacks LE blocks but at the end, uses average pooling $\rightarrow$ dropout $\rightarrow$ decoding layer $\rightarrow$ softmax
 - Eg. $v_\theta(t, x_0)_i = \phi_{\text{out}}([g_0 \text{ or } t], g^L, x_i^L, x_i^0)$
- Have multiple message-passing steps in the same layer, i.e., repeat $x_i^{l+1} = x_i^l + \gamma_l \sum_{j \in N, j \neq i} \phi_x(g^0, g^{l+1}, m_{ij}) x_j^l$ inside a layer
- Linear combination with differences between 4-vectors based on E(3) GNN [2]

$$x_i^{l+1} = x_i^l + \sum_{j \neq i} \phi_x(g^0, g^{l+1}, m_{ij}) \frac{\mathbf{x}_i^1 - \mathbf{x}_j^1}{\|\mathbf{x}_i^1 - \mathbf{x}_j^1\| + 1}$$

Potential refinements

- Using simple linear conditional flow objective (theoretically best for training stability), could try more sophisticated affine flows [3] or DDIM (theoretically less stable)
- Currently 16 Euler integration steps, 128-512 is more typical in published work (tend to get diminishing returns)
- Increased batch size: currently restricted to ≈ 50 with $\mathcal{O}(N^2)$ network on particle clouds with $N_C = 150$ (on a 24 GB device)

But none of these should be the difference between trivial and meaningful output

Pending fixes

- Global features (jet η , m , p_T , $\#$ constituents) are
 1. Produced with a non-LE normalizing flow
 2. Are passed through an MLP (with time embedding and one-hot encoded jet type) in the main flow matching network to produce the global embedding, which breaks LEPotential fix: only generate LE feature ($\#$ constituents) as part of the global embedding
- Discussion from last week on the initial distribution - using a simple normal right now for debugging (not LE)

New architecture I

1. Embed scalar time value to random Fourier features through two fully connected layers with 32 and 64 nodes (following [1]) to produce t_{emb}
2. Pass jet information (one-hot encoded jet type and scalar number of jet constituents N_C) through 2 fully connected layers to 64-dimensional embedding g_0
3. Pass this through a fully connected layer of size 64 to produce the initial global embeddings g^0
4. Concatenate g^0 with features $x_1^0 \dots x_N^0$ initialized from the initial distribution p_0 .
5. Now, we use L Lorentz-group equivariant layers.
 - 5.1 The input to the $l + 1$ -th layer is g^l and $x_1^l \dots x_N^l$.
 - 5.2 Calculate messages $m_{ij}^l = \phi_e^l(\psi(\|x_i^l - x_j^l\|), \psi(\langle x_i^l, x_j^l \rangle))$
 - 5.3 **Update the global embedding**

$$g_{l+1} = \phi_g^l(g^l, \alpha_l \sum_{i,j \in [N]} \phi_m^l(m_{ij}) m_{ij}^l)$$

New architecture II

5.4 Feed in the updated global embedding as well as the original embedding to update the graph embeddings

$$x_i^{l+1} = x_i^l + \gamma_l \sum_{j \in N, j \neq i} \phi_x^l(g^0, g^{l+1}, m_{ij}) x_j^l$$

5.5 Pass the results g^{l+1} and x_i^{l+1} to the next layer

6. **Final output** $v_\theta(t, x^0)_i = x_i^L$ ~~x_i^0~~

- ϕ = MLP with LeakyRELU activations except for ϕ_x , which uses tanh (for expressiveness in linear combinations)
- α initialized to $\frac{1}{N^2}$, γ initialized to $\frac{1}{N}$

References I

- [1] Lemeng Wu et al. “Fast Point Cloud Generation with Straight Flows”. In: *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. Vancouver, BC, Canada: IEEE, June 2023, pp. 9445–9454. ISBN: 979-8-3503-0129-8. DOI: 10.1109/CVPR52729.2023.00911. (Visited on 06/23/2025).
- [2] Victor Garcia Satorras, Emiel Hoogeboom, and Max Welling. *E(n) Equivariant Graph Neural Networks*. Feb. 2022. DOI: 10.48550/arXiv.2102.09844. arXiv: 2102.09844 [cs]. (Visited on 06/25/2025).
- [3] Yaron Lipman et al. *Flow Matching Guide and Code*. Dec. 2024. DOI: 10.48550/arXiv.2412.06264. arXiv: 2412.06264 [cs]. (Visited on 06/21/2025).